



ZAP by Checkmarx Scanning Report

Site: <http://localhost:8000>

Generated on Sun, 14 Jun 2026 12:42:19

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	2
Informational	3
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP errors logged - see the zap. log file for details	5
Low	Warning		ZAP warnings logged - see the zap. log file for details	1
Info	Informational		Percentage of network failures	1 %
Info	Informational	http://localhost:8000	Percentage of responses with status code 2xx	28 %
Info	Informational	http://localhost:8000	Percentage of responses with status code 4xx	68 %
Info	Informational	http://localhost:8000	Percentage of responses with status code 5xx	3 %

Info	Informational	http://localhost:8000	Percentage of endpoints with content type text/css	6 %
Info	Informational	http://localhost:8000	Percentage of endpoints with content type text/html	93 %
Info	Informational	http://localhost:8000	Percentage of endpoints with method GET	100 %
Info	Informational	http://localhost:8000	Count of total endpoints	15
Info	Informational	http://localhost:8000	Percentage of slow responses	16 %

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	Systemic
Missing Anti-clickjacking Header	Medium	Systemic
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	Systemic
X-Content-Type-Options Header Missing	Low	Systemic
Authentication Request Identified	Informational	2
Information Disclosure - Sensitive Information in URL	Informational	5
User Controllable HTML Element Attribute (Potential XSS)	Informational	4

Alert Detail

Medium	Content Security Policy (CSP) Header Not Set
Description	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	http://localhost:8000/
Node Name	http://localhost:8000/

Method	GET
Attack	
Evidence	
Other Info	
URL	http://localhost:8000/catalogue.html
Node Name	http://localhost:8000/catalogue.html
Method	GET
Attack	
Evidence	
Other Info	
URL	http://localhost:8000/index.html
Node Name	http://localhost:8000/index.html
Method	GET
Attack	
Evidence	
Other Info	
URL	http://localhost:8000/robots.txt
Node Name	http://localhost:8000/robots.txt
Method	GET
Attack	
Evidence	
Other Info	
URL	http://localhost:8000/sitemap.xml
Node Name	http://localhost:8000/sitemap.xml
Method	GET
Attack	
Evidence	
Other Info	
Instances	Systemic
Solution	Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html https://www.w3.org/TR/CSP/ https://w3c.github.io/webappsec-csp/ https://web.dev/articles/csp https://caniuse.com/#feat=contentsecuritypolicy https://content-security-policy.com/

CWE Id	693
WASC Id	15
Plugin Id	10038

Medium	Missing Anti-clickjacking Header
---------------	---

Description	The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.
-------------	---

URL	http://localhost:8000/
-----	---

Node Name	http://localhost:8000/
-----------	------------------------

Method	GET
--------	-----

Attack	
--------	--

Evidence	
----------	--

Other Info	
------------	--

URL	http://localhost:8000/catalogue.html
-----	---

Node Name	http://localhost:8000/catalogue.html
-----------	--------------------------------------

Method	GET
--------	-----

Attack	
--------	--

Evidence	
----------	--

Other Info	
------------	--

URL	http://localhost:8000/contact.html
-----	---

Node Name	http://localhost:8000/contact.html
-----------	------------------------------------

Method	GET
--------	-----

Attack	
--------	--

Evidence	
----------	--

Other Info	
------------	--

URL	http://localhost:8000/index.html
-----	---

Node Name	http://localhost:8000/index.html
-----------	----------------------------------

Method	GET
--------	-----

Attack	
--------	--

Evidence	
----------	--

Other Info	
------------	--

URL	http://localhost:8000/profile.html
-----	---

Node Name	http://localhost:8000/profile.html
-----------	------------------------------------

Method	GET
--------	-----

Attack	
--------	--

Evidence	
----------	--

--	--

Other Info	
Instances	Systemic
Solution	Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
CWE Id	1021
WASC Id	15
Plugin Id	10020

Low	Server Leaks Version Information via "Server" HTTP Response Header Field	
Description	The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate attackers identifying other vulnerabilities your web/application server is subject to.	
URL	http://localhost:8000/	
Node Name	http://localhost:8000/	
Method	GET	
Attack		
Evidence	SimpleHTTP/0.6 Python/3.11.1	
Other Info		
URL	http://localhost:8000/index.html	
Node Name	http://localhost:8000/index.html	
Method	GET	
Attack		
Evidence	SimpleHTTP/0.6 Python/3.11.1	
Other Info		
URL	http://localhost:8000/robots.txt	
Node Name	http://localhost:8000/robots.txt	
Method	GET	
Attack		
Evidence	SimpleHTTP/0.6 Python/3.11.1	
Other Info		
URL	http://localhost:8000/sitemap.xml	
Node Name	http://localhost:8000/sitemap.xml	
Method	GET	
Attack		

Evidence	SimpleHTTP/0.6 Python/3.11.1
Other Info	
URL	http://localhost:8000/style.css
Node Name	http://localhost:8000/style.css
Method	GET
Attack	
Evidence	SimpleHTTP/0.6 Python/3.11.1
Other Info	
Instances	Systemic
Solution	Ensure that your web server, application server, load balancer, etc. is configured to suppress the "Server" header or provide generic details.
Reference	https://httpd.apache.org/docs/current/mod/core.html#servertokens https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff648552(v=pandp.10) https://www.troyhunt.com/shhh-dont-let-your-response-headers/
CWE Id	497
WASC Id	13
Plugin Id	10036

Low	X-Content-Type-Options Header Missing
Description	The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.
URL	http://localhost:8000/
Node Name	http://localhost:8000/
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://localhost:8000/catalogue.html
Node Name	http://localhost:8000/catalogue.html
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://localhost:8000/contact.html
Node	

Name	http://localhost:8000/contact.html
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://localhost:8000/index.html
Node Name	http://localhost:8000/index.html
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://localhost:8000/style.css
Node Name	http://localhost:8000/style.css
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
Instances	Systemic
Solution	Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages. If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application /web server to not perform MIME-sniffing.
Reference	https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85) https://owasp.org/www-community/Security_Headers
CWE Id	693
WASC Id	15
Plugin Id	10021

Informational	Authentication Request Identified
Description	The given request has been identified as an authentication request. The 'Other Info' field contains a set of key=value lines which identify any relevant fields. If the request is in a context which has an Authentication Method set to "Auto-Detect" then this rule will change the authentication to match the request identified.
URL	http://localhost:8000/profile.html?bio=CS+student%2C+loves+science+fiction+and+history+books.&fullname=Amina+Khalid&password=ZAP&username=ZAP
Node Name	http://localhost:8000/profile.html (bio,fullname,password,username)

Method	GET
Attack	
Evidence	password
Other Info	userParam=username userValue=ZAP passwordParam=password referer=http://localhost:8000/profile.html?password=ZAP&username=ZAP
URL	http://localhost:8000/profile.html?password=ZAP&username=ZAP
Node Name	http://localhost:8000/profile.html (password,username)
Method	GET
Attack	
Evidence	password
Other Info	userParam=username userValue=ZAP passwordParam=password referer=http://localhost:8000/profile.html
Instances	2
Solution	This is an informational alert rather than a vulnerability and so there is nothing to fix.
Reference	https://www.zaproxy.org/docs/desktop/addons/authentication-helper/auth-req-id/
CWE Id	
WASC Id	
Plugin Id	10111

Informational	Information Disclosure - Sensitive Information in URL
Description	The request appeared to contain sensitive information leaked in the URL. This can violate PCI and most organizational compliance policies. You can configure the list of strings for this check to add or remove values specific to your environment.
URL	http://localhost:8000/contact.html?email=zaproxy%40example.com&message=Zaproxy+alias+impedit+expedita+quisquam+pariatur+exercitationem.+Nemo+rerum+eveniet+dolores+rem+quia+dignissimos.&name=ZAP&subject=Zaproxy+dolore+alias+impedit+expedita+quisquam.
Node Name	http://localhost:8000/contact.html (email,message,name,subject)
Method	GET
Attack	
Evidence	zaproxy@example.com
Other Info	The URL contains email address(es).
URL	http://localhost:8000/profile.html?bio=CS+student%2C+loves+science+fiction+and+history+books.&fullname=Amina+Khalid&password=ZAP&username=ZAP
Node Name	http://localhost:8000/profile.html (bio,fullname,password,username)
Method	GET
Attack	
Evidence	password
Other Info	The URL contains potentially sensitive information. The following string was found via the pattern: pass password
URL	http://localhost:8000/profile.html?bio=CS+student%2C+loves+science+fiction+and+history+books.&fullname=Amina+Khalid&password=ZAP&username=ZAP
Node	

Name	http://localhost:8000/profile.html (bio,fullname,password,username)
Method	GET
Attack	
Evidence	username
Other Info	The URL contains potentially sensitive information. The following string was found via the pattern: user username
URL	http://localhost:8000/profile.html?password=ZAP&username=ZAP
Node Name	http://localhost:8000/profile.html (password,username)
Method	GET
Attack	
Evidence	password
Other Info	The URL contains potentially sensitive information. The following string was found via the pattern: pass password
URL	http://localhost:8000/profile.html?password=ZAP&username=ZAP
Node Name	http://localhost:8000/profile.html (password,username)
Method	GET
Attack	
Evidence	username
Other Info	The URL contains potentially sensitive information. The following string was found via the pattern: user username
Instances	5
Solution	Do not pass sensitive information in URIs.
Reference	
CWE Id	598
WASC Id	13
Plugin Id	10024

Informational	User Controllable HTML Element Attribute (Potential XSS)
Description	This check looks at user-supplied input in query string parameters and POST data to identify where certain HTML attribute values might be controlled. This provides hot-spot detection for XSS (cross-site scripting) that will require further review by a security analyst to determine exploitability.
URL	http://localhost:8000/catalogue.html?category=cs&comment=Zaproxy+dolore+alias+impedit+expedita+quisquam.&name=ZAP&q=ZAP
Node Name	http://localhost:8000/catalogue.html (category,comment,name,q)
Method	GET
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://localhost:8000/catalogue.html?category=cs&comment=Zaproxy+dolore+alias+impedit+expedita+quisquam.&name=ZAP&q=ZAP appears to include user input in: a(n) [option] tag [value] attribute The user input found was: category=cs The user-controlled value was: cs
URL	http://localhost:8000/catalogue.html?category=cs&q=ZAP

Node Name	http://localhost:8000/catalogue.html (category,q)
Method	GET
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://localhost:8000/catalogue.html?category=cs&q=ZAP appears to include user input in: a(n) [option] tag [value] attribute The user input found was: category=cs The user-controlled value was: cs
URL	http://localhost:8000/profile.html?bio=CS+student%2C+loves+science+fiction+and+history+books.&fullname=Amina+Khalid
Node Name	http://localhost:8000/profile.html (bio,fullname)
Method	GET
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://localhost:8000/profile.html?bio=CS+student%2C+loves+science+fiction+and+history+books.&fullname=Amina+Khalid appears to include user input in: a(n) [input] tag [value] attribute The user input found was: fullname=Amina Khalid The user-controlled value was: amina khalid
URL	http://localhost:8000/profile.html?bio=CS+student%2C+loves+science+fiction+and+history+books.&fullname=Amina+Khalid&password=ZAP&username=ZAP
Node Name	http://localhost:8000/profile.html (bio,fullname,password,username)
Method	GET
Attack	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://localhost:8000/profile.html?bio=CS+student%2C+loves+science+fiction+and+history+books.&fullname=Amina+Khalid&password=ZAP&username=ZAP appears to include user input in: a(n) [input] tag [value] attribute The user input found was: fullname=Amina Khalid The user-controlled value was: amina khalid
Instances	4
Solution	Validate all input and sanitize output it before writing to any HTML attributes.
Reference	https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
CWE Id	20
WASC Id	20
Plugin Id	10031